Machine Learning System Design interview
MY CURRICULUM
1 Introduction and Overview
2 Metrics
3 Ads Recommendation System
4 Newsfeed Ranking System
5 Visual Search System
6 Harmful Content Detection
7 Video Recommendation System
8 Similar Listings
9 Similar Listings on Marketplace
10 Hashtag Recommendation
11 Personalized News Feed
12 People You May Know

09
Similar Listings on Vacation Rental Platforms

Recommending items similar to those a user is currently viewing is a key technology that allows people to discover attractively relevant content on large platforms. In this chapter, we design a similar listings recommendation system, focusing on vacation rental platforms, which frequently recommends similar properties to guests.

Figure 01: Recommended similar listings

Clarifying Requirements

Here is a typical interview conversation between a candidate and an interviewer.

Candidate: Can I assume the business objective is to increase the number of bookings?
Interviewer: Yes.
Candidate: What is the definition of "similarity"? Are the recommended listings expected to be similar to the listing that a user is currently viewing?
Interviewer: Great question. Let's start with something as simple when they are in the same neighborhood, city, price range, and have similar amenities. The recommended listings generally personalized to users?
Candidate: Let's keep it simple for now. We can add personalization and complex similarity metrics later. Do we have enough differentiating data and capabilities to support users' browsing behavior? For simplicity, let's assume we have logged it and enough data for users exists.
Candidate: How many listings are available on the platform?
Interviewer: Assume 6 million listings.
Candidate: Do we have a cold-start listing problem?
Interviewer: Great question. Yes, this is a real-world problem we must address. We have multiple listing providers. The model doesn't leverage users' attributes, such as age or location, or a listing's attributes, like price and location, at all.
Candidate: Do we have enough data for training models to address the similar listings problem?
Interviewer: Let's assume there are enough data to address recommendations only after listings booking. During this time, the model should leverage other signals, such as listing's attributes.

Let's summarize the requirements. Most similarity recommendation systems at Airbnb-like platforms recommend similar listings, or a specific listing that a user is currently viewing, and the product is centered on the goal of similar listings that users likely to click or book. The recommended listings should be the same for both logged-in and logged-out users. Since there is enough to define a strong baseline, we can add personalization later. We have enough data to support recommendations during training. Here, we'll take a cold-start approach and leverage content-based recommendations.

Business Goals and Metrics

A session-based recommendation system aims to predict the next item, given a sequence of recent items browsed by a user. In contrast, session-based recommendations rely on current sessions to make recommendations. The accuracy depends on the current and recent interactions, not their precise distances.

Figure 02: A similar listings system's input-output

Choosing the right ML category

Most recommendations rankings are a users' historical interactions to understand their long-term interests. However, such recommendation systems may be quick to recommend items similar to those a user is currently viewing as input and their outputs a ranked list of listings, ensuring the probability of the user clicking on them.

Figure 03: Similar listings system's input-output

How do sessions-based and traditional recommendation systems compare?

In traditional recommendation systems, users' interests are context-independent and we'll change buy frequently. In session-based recommendation systems, user's interests are context-dependent and we'll change buy frequently. Examples of such contexts are users' genuine interests, or current intent. In session-based recommendation systems, the underlying user's genuine interest and intent, based on their recent browsing history.

Session-based recommendations are a subset of session-based. Session-based recommendation systems to learn from embeddings using two sequences of user's browsing history. For example, Instagram Explore recommendation system to power the "Explore" feature. ID-based learning learns embeddings to power by similar listings features (ID) and prediction (ID) using a similar approach to learn meaningful word embeddings.

Session-based recommendations learn the "similarity" between items in a session based recommendations. We build the systems to training a model to predict the next item, given a sequence of recent items browsed by a user. Session-based recommendations rely on current sessions to make recommendations. The accuracy depends on the current and recent interactions, not their precise distances.

Figure 04: Similar listings in the embedding space

Data Preparation

Data engineering

The following data sources:

- Listings
- Users
- User-listing interactions

Users

A simplified user data schema is shown below.

ID	Username	Age	Gender	City	Country	Language	Time zone
1	Jack	28	M	Paris	FR	FR	Europe/Paris

Table 01: User data schema

Listings

Listing data contains attributes related to each listing, such as price, number of beds, host ID, etc. Table 02 shows a simple example of what the listing data might look like.

ID	Host ID	Price	Num. Beds	Name	Listing Type	City	Made	Max guests
1	1	100	1	Cozy studio	Studio	Paris	2020-01-01	2
2	2	80	0.5	Nice	Room	NY	1	1
3	3	200	2	Beautiful house	Entire home	Boston	2	4

Table 02: Listings data schema

User-listing interactions

Table 03 shows user-listing interactions such as impressions, clicks, and bookings.

ID	User ID	Listing ID	Position of the listing in the displayed list	Interaction type	Source	Timestamp
1	10	20	1	Click	Search feature	1650100000
2	5	10	5	Book	Similar listing feature	1650100001

Table 03: User-listing interaction data

Feature engineering

As described in the "Frame the Problem as ML Task" section, the model only utilizes users' browsing history during training. Other features are used only in the ranking model, as we'll see later.

In this chapter, the remaining sections are called "feature engineering" and focused on a sequence of similar listing features, inspired by an eventually bounded listing, without interruption. Figure 05 shows an example of a search session, where the user's session started when the user clicked on a listing and ended when the user eventually booked.

Figure 05: A search session

In the feature engineering step, we extract search sessions from the interaction data. Table 04 shows a simple example of the search sessions.

Session ID	Clicked listing IDs	Eventually booked listing ID
1	10,45,67	89
2	9,50,23,14,34	3
3	12	12

Table 04: A search session data

Model Development

Model selection

A block network is the standard method in learn embeddings. Choosing a good architecture for the neural network depends upon the input features, such as sparse or dense features, the amount of training data, the complexity of the relationship between the hyperparameters associated with neural network architecture, such as the number of neurons, layers, activation functions, etc., or is one hyperparameter and choosing the architecture that performs best is a time-consuming process. We'll use a two-tower neural network for learning embeddings.

Model training

As shown in Figure 06, for a given input listing, the model's job is to predict listings within the input listing's context.

Figure 07: Contrastive pairs and negative listing pairs

Choosing the loss function

Loss function measures the agreement between the ground-truth label and the predicted probability. If two listings form a positive pair, the ground-truth label is 1, and if the two listings form a negative pair, the ground-truth label is 0. Since the ground-truth is binarized, we use the binary cross-entropy loss function.

1. Compute the distance (d_{ij}) and product between two embeddings.
2. Pass the product through a sigmoid function to convert the product to probability range between 0 and 1.
3. Use binary cross-entropy standard loss function to measure the loss between the predicted probability and the ground-truth label.

Figure 08 shows the loss calculation steps.

Figure 08: Loss calculation steps

The loss can be represented by the following formula:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \cdot \log \sigma(d_{ij}) + (1 - y_i) \cdot \log(1 - \sigma(d_{ij}))\right]$$

Where:

- N is a partial listing, a pre-positive listing (or concurrent with) or a contrast), and y_i is a negative listing (did not or never with) or a positive listing.
- d_{ij} represents the embedding vector of the listing i.
- d_{ij} represents the embedding vector of the listing j.
- d_{ij} represents the embedding vector of similarity listing i.
- d_{ij} is a positive or all pairs y_i which represents (central listing, contrast listing) replace whose vectors are being pushed closer together.
- d_{ij} is a negative or all pairs y_i which represents (central listing, contrast listing) replace whose vectors are being pushed further away from each other.

The first summation represents the loss over positive pairs and the second summation computes the loss over negative pairs.

Can we improve the loss function to learn better embeddings?

The loss function described in the previous section treats all pairs equally and has two shortcomings. First, during training, the embeddings of the central listing is pushed closer to the embeddings in its contrast, but can bias towards the embedding of the eventually booked listing. This leads to pushing the embeddings of the other contrasting listings farther away, but at a positively eventually booked listing. This leads to pushing the embeddings of the other contrasting listings farther away.

Second, the negative pairs generated center mainly comprise listings from different regions, since they are random remotely. Consequently, the model easily learns to distinguish such a random negative pair. The embedding learned by the embeddings that do not help learn meaningful listings, those that have not co-occurred in contrast, but are from same region.

Let's address these shortcomings.

Using the eventually booked listing as a global contrast

To address the first shortcoming, for each pair of central listing and contrast listing, we treat the eventually booked listing as a global contrast pairing for the central listing. As the central listing is pushed away from its contrast, we also simultaneously push the eventually booked listing always remains in the global contrast, and is used to update the central listing vector.

Figure 09: Pushing the eventually booked listing in the positive pairs

To use the eventually booked listing as a global contrast during training, we add pairs of , central listing, eventually booked listing, to a loss training data and label them as positive. This allows the model to push the embeddings of the eventually booked listing closer to the central listing, and simultaneously pushes the central listing and its contrast further apart.

Add negative pairs from same region to the training data

As the window slides, we choose a listing from the same neighborhood as the central listing, which is not within the current listing's context. We randomly pick a negative pair and it is our training data. Let's use the updated loss function that computes rarely added training data.

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \cdot \log \sigma(d_{ij}) + (1 - y_i) \cdot \log(1 - \sigma(d_{ij}))\right]$$

Where:

- N represents the embedding vector of the eventually booked listing i
- y_{book} are pairs of i, b that represent (central listing, booked listing) replace whose vectors are being pushed closer to each other.
- y_{neg} are pairs of i, neg that represent (central listing, same region listing) replace whose vectors are being pushed away from each other.

The fourth summation computes the loss over rarely added negative pairs from the same region. It enforces the model to push these embeddings away from each other.

Offline metrics

During the model development phase, we also need to measure the output quality of the model and compare the rarely added training data. Let's use the following offline metrics to measure the output quality of the model. Let's use the rarely added eventually booked listing, stored in the latest year of data, Let's create a rarely added training data set of eventually booked listings and discuss how to more ideal.

The average rank of the eventually booked listing. Let's look at an example to understand the metric. Figure 10 shows a search session, where the user booked listing 3 and the model ranked the eventually booked listing at rank 2. The first listing is skip the user caused the L_{ij} The next two are listings the user clicked, and the last one is the listing that the user eventually booked.

Figure 10: A session in ranked by a model

We can the model to compute the embeddings between the central listing and other listings in the embedding space. Since embeddings are computed, the listings are ranked. The cosine similarity between the embeddings is used that is the similarity and the closer to 1 is more similar. Let's measure how the model, so we can see in Figure 10 it, the two booked listings rank at rank 1 and rank 2 respectively.

If the model ranks the eventually booked listing highly, it indicates the learned embeddings can place the eventually booked listing spatially close to the corresponding listings the user clicked of the eventually booked listings across all the sessions in the validation dataset, it's unique the value of this metric.

Online metrics

According to the requirements, the business objective is to increase the number of bookings. Here are some options for online metrics:

- Click through rate (CTR)
- Session book rate

CTR is only relevant may often applying user filters and ranking preferences. Let's use session book rate.

Session book rate is the number of sessions booked divided by the number of sessions.

$$\text{Session book rate} = \frac{\text{Number of sessions booked listing impressions}}{\text{Number of sessions}}$$

Session book rate is the number of sessions booked listing impressions divided by the number of sessions.

Session book rate is the number of sessions booked listing impressions divided by the number of sessions.

Relevant neighbor services

To recommend similar listings, we need to measure the relevance between the embeddings of the currently viewing listing and the recommended similar listings. Let's use the following metrics to measure the relevance between the embeddings of the currently viewing listing and the recommended similar listings.

Remember from the requirements that we have five million listings on the platform. Computing similarities for this many listings takes time and cannot be done in real-time. Therefore, we need an approximate nearest neighbor method to speed up the search.

Other Talking Points

Here is a list of other topics that we could talk about, if time permits.

- What is positive bias, and how to address it (1).
- How to address the cold-start listing problem (2) (session with (8), and from session walks with random (RWR) can be used to recommend similar listings (3)
- How to personalize the results of a session based similarity recommendation system by considering users' longer term interests (4)
- Dwell time prediction (5)
- Guest that seasonality greatly affects vacation rentals, how should we incorporate seasonality into our similar listings system (6)
- ...

References

1. Instagram's Explore recommendation system. https://engineering.fb.com/2019/03/14/core-data/applying-personalization-to-instagram-explore-recommendations/
2. Listing embeddings in search ranking. https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-personalized-search-ranking-25f145f0f665
3. Pixie: https://arxiv.org/abs/1702.07969
4. Negative sampling techniques. https://towardsdatascience.com/how-exactly-negative-sampling-works-f0ea0f09c0f9
5. Dwell time prediction. https://www.cs.cornell.edu/~shuochen/papers/kdd18-dwell.pdf
6. Seasonality. https://en.wikipedia.org/wiki/Seasonality#Time_series_data
7. Random walk with restarts. https://www.cs.cmu.edu/~christos/PUBLICATIONS/icdm10-fast-rwr.pdf
8. Session-based recommendations with recurrent neural networks. https://arxiv.org/abs/1511.07939

Mark as Complete

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.

Ask Alex